

Extração e Validação de Dados em Arquivos Semi-Estruturados utilizando Expressões Regulares: Uma Abordagem Prática

Ana Letícia de Oliveira Calandrini, Juliana Rodrigues Pereira
Lucas Santos Diniz, Marceley Lopes dos Anjos, Yuri Gabriel Cardoso Delgado

¹Instituto de Ciências Exatas e Naturais – Universidade Federal do Pará (UFPA)
Caixa Postal 66075-110 – Rua Augusto Corrêa, 01, Bairro – Guamá – Brazil

{ana.leticia, juliana.pereira, lucas.diniz, marceley.dosanjos,
yuri.delgado}@icen.ufpa.br

Abstract. *In real-world applications, textual information is frequently unstructured, containing noise, inconsistencies, and multiple representation formats. This paper presents the development of an automated system for textual inspection based on Regular Expressions (Regex), written in Python. The system is capable of reading different file types (free text, logs, chats, and CSVs), extracting specific patterns (such as emails, CPFs, phones, and dates), and structurally validating them. Finally, the extracted data is organized and exported in JSON format, alongside quantitative statistics, demonstrating the practical efficiency of Formal Languages theory in data mining.*

Resumo. *Em aplicações reais, informações textuais frequentemente encontram-se desestruturadas, contendo ruídos, inconsistências e múltiplos formatos de representação. Este artigo descreve o desenvolvimento de um sistema automatizado de inspeção textual baseado em Expressões Regulares (Regex), construído em Python. O sistema é capaz de realizar a leitura de diferentes tipos de arquivos (texto livre, logs, chats e CSVs), extrair padrões específicos (como e-mails, CPFs, telefones e datas) e validá-los de forma estrutural. Por fim, os dados extraídos são organizados e exportados em formato JSON, acompanhados de estatísticas quantitativas, demonstrando a eficiência prática da teoria de Linguagens Formais na mineração de dados.*

1. Introdução

Na era da informação, sistemas computacionais geram e armazenam volumes massivos de dados diariamente. No entanto, uma parcela significativa desses dados encontra-se em formatos semi-estruturados ou desestruturados, como logs operacionais, históricos de mensagens e exportações de bancos de dados mal formatados. A presença de ruídos e inconsistências nestes arquivos torna a análise manual inviável, exigindo o uso de ferramentas automatizadas para a extração e validação de conhecimento. Neste contexto, a teoria das Linguagens Formais e Autômatos fornece a base matemática e computacional para a resolução desse problema. As Expressões Regulares (Regex), originárias dos estudos de Stephen Kleene, constituem um mecanismo poderoso e eficiente para o reconhecimento de cadeias de caracteres pertencentes a linguagens regulares específicas.

Através delas, é possível mapear padrões sintáticos complexos em grandes volumes de texto. O presente trabalho foi elaborado seguindo essas diretrizes na atividade proposta dentro da disciplina de Linguagens Formais e Autômatos pela Universidade Federal do Pará. Tem como objetivo descrever o desenvolvimento de um sistema de inspeção textual em Python que utiliza Expressões Regulares para processar múltiplos arquivos de entrada.

2. Objetivos

O presente projeto tem como objetivo desenvolver um sistema automatizado de inspeção e análise textual, fundamentado nos conceitos de Linguagens Formais e Autômatos, utilizando Expressões Regulares (Regex) como mecanismo principal para reconhecimento de padrões sintáticos em arquivos semi-estruturados e desorganizados.

O sistema foi projetado para realizar a leitura e o processamento automatizado de diferentes tipos de arquivos, identificando suas características estruturais e classificando o conteúdo de acordo com seu formato, como registros de conversas, logs de sistema, arquivos de texto livre e arquivos CSV inconsistentes.

Além disso, o projeto busca extrair informações relevantes presentes nos dados analisados, tais como datas, horários, e-mails, números de telefone, CPFs, URLs, valores monetários e nomes próprios, aplicando técnicas de validação para determinar a consistência e a conformidade dessas informações com padrões previamente definidos.

Outro objetivo fundamental consiste na detecção de inconsistências estruturais, especialmente em arquivos do tipo CSV, verificando problemas como divergência no número de colunas e presença de campos vazios, contribuindo para a identificação de falhas nos dados de entrada.

Por fim, o sistema visa organizar as informações coletadas de maneira estruturada, gerando arquivos no formato JSON para facilitar armazenamento, manipulação e posterior análise dos dados, além de produzir estatísticas quantitativas referentes às ocorrências identificadas, permitindo uma visão consolidada do conteúdo processado.

3. Materiais e Métodos

O sistema foi desenvolvido utilizando a linguagem de programação Python (versão 3.10+), escolhida por sua robustez e vasta adoção em tarefas de processamento de texto e ciência de dados. O ambiente de desenvolvimento foi heterogêneo: alguns integrantes utilizaram o sistema operacional Linux, enquanto outros operaram em Windows. A codificação foi conduzida através do editor Visual Studio Code e, visando otimizar a colaboração à distância, utilizou-se a extensão *Live Share*, que viabilizou a edição simultânea do projeto. Complementarmente, a comunicação e definição de metas, objetivos e acompanhamento do projeto foi realizada utilizando a plataforma Discord.

A arquitetura do projeto foi desenhada em formato de pipeline de processamento modular. As principais bibliotecas padrão utilizadas foram:

- **re:** Para a compilação e execução dos motores de Expressões Regulares (Reconhecimento Léxico e Validação Sintática).
- O regex utilizado é derivado em Perl (PCRE), ou seja, ele possui sintaxe e recursos parecidos, porém há limitações e comportamentos próprios, como:

1. O PCRE e a biblioteca de terceiros regex do Python suportam *lookbehinds* de tamanho variável, enquanto o módulo nativo exige tamanhos fixos.
 2. O PCRE e o Perl suportam recursão direta, que no Python nativo é emulada de forma limitada.
 3. O Python implementou subpadrões nomeados em formato próprio, que o PCRE e o Perl passaram a suportar posteriormente.
- **json**: Para a estruturação dos dados extraídos e persistência em arquivos.
 - **pathlib**: Para a manipulação segura e multiplataforma de caminhos de diretórios (ex: acesso à pasta `./assets/` e gravação em `./resultados.json/`).

3.1. Modularização do Código

A modularização do código foi adotada para garantir a organização, facilitar a manutenção e aplicar o princípio de separação de responsabilidades dentro da arquitetura de software. Em vez de concentrar toda a lógica em um único script extenso e difícil de debugar, a divisão modular permite que cada parte do sistema cuide de uma tarefa específica, tornando o código muito mais legível e escalável para futuras implementações ou correções.

O sistema foi dividido nos seguintes módulos:

1. **config.py**: Arquivo base que armazena as configurações estáticas e os padrões léxicos do sistema. Apresenta as seguintes funções/variáveis:
 - (a) Diretórios (DIRETORIO_ASSETS, DIRETORIO_JSON): Define os caminhos de entrada e saída de dados de forma segura usando a biblioteca `pathlib`.
 - (b) Arquivos de Leitura (ARQUIVOS_OBRIGATORIOS): Lista os quatro arquivos exatos de teste que o sistema vai processar (texto livre, logs, chat e CSV).
 - (c) Identificadores de Classe (REGEX_CHAT, REGEX_LOG): Expressões usadas para classificar automaticamente qual é o tipo do arquivo lido identificando palavras-chave ou padrões de tempo.
 - (d) Extratores e Validadores (REGEX_EMAIL, REGEX_CPF, REGEX_NOME, etc.): Guardam os padrões de busca (Regex) para capturar os dados nos textos e validá-los estruturalmente, separando ocorrências válidas das inválidas/mal formatadas.
2. **analyzer.py**: Funciona como o motor de processamento bruto do sistema, contendo as lógicas de leitura e mineração, que apresenta as seguintes funções:
 - (a) `inspecao(arquivo)`: Percorre os arquivos ignorando linhas vazias e contabilizando o total de linhas úteis.
 - (b) `verificaTipo(arquivo)`: O conteúdo é classificado utilizando heurísticas baseadas no conteúdo e na extensão do arquivo.
 - (c) Identifica as categorias "Chat", "Log", "CSV" ou, por eliminação, "Texto Livre".
 - (d) `validarCSV(arquivo)`: Verifica se a quantidade de colunas em cada linha corresponde ao cabeçalho e detecta a presença de campos vazios.
 - (e) `extrairOcorrencias(arquivo)`: Múltiplas expressões regulares são aplicadas para capturar padrões textuais. Ela varre cada linha, busca os padrões (emails, CPFs, datas, etc.), chama os métodos de validação (`validators.py`) para categorizar como válido ou inválido, registra os dados encontrados e atualiza as contagens estatísticas do sistema.

- (f) `mostrarAmostra(arquivo)`: Retorna uma amostra reduzida garantindo a visibilidade da leitura sem sobrecarregar o console.
 - (g) `adicionarAtributo(dadosLinha, tipo, valor, classificacao)`: Função auxiliar interna para padronizar a forma como cada dado encontrado é estruturado no dicionário antes de ser exportado.
3. **validators.py**: Este módulo é dedicado exclusivamente à segunda etapa da estratégia de validação descrita no seu artigo. Ele aplica a validação estrutural aos dados, atestando a integridade sintática das informações capturadas. Apresenta as seguintes funções:
- (a) `validarEmail(email)`, `validarTelefone(telefone)`, `validarCPF(cpf)` e `validarURL(url)`: Estas funções recebem as cadeias de texto que foram capturadas de forma genérica na primeira etapa de varredura. Elas pegam esse dado e o submetem a uma segunda Expressão Regular restritiva (importada do `config.py`). Todas elas utilizam o método `re.fullmatch()` para confirmar se a string corresponde exata e integralmente ao padrão definido. O resultado é um valor booleano que o sistema usa para classificar definitivamente aquela ocorrência como "válida" ou "inválida".
4. **stats.py**: É o módulo responsável pelo controle das métricas e pela geração da Análise Quantitativa Global. Apresenta as seguintes estruturas:
- (a) `estatisticas` (Dicionário): Um dicionário global em memória que armazena a distribuição de todas as ocorrências mapeadas pelo sistema. Ele contabiliza os totais de cada padrão extraído (e-mails, CPFs, URLs, inconsistências de CSV, etc.), separando a quantidade de dados válidos e inválidos, e organizando os números por arquivo de origem.
 - (b) `atualizarEstatisticaArquivo(tipo, arquivo)`: Uma função auxiliar simples que incrementa a contagem de um padrão específico (como "emails" ou "nomes") diretamente dentro do registro do arquivo onde foi encontrado. É esta função que alimenta os dados exportados no relatório final `estatisticas.json`.
5. **main.py**: Atua como o arquivo principal e ponto de entrada do sistema. Ele é responsável pela junção dos outros módulos, gerenciando o fluxo de execução do programa do início ao fim e controlando a interface de linha de comando. Apresenta as seguintes funções principais:
- (a) `inspecionar()`: É o fluxo de execução principal do sistema. Ele garante que a pasta de resultados exista, itera sobre a lista de arquivos obrigatórios do `config.py`, aciona as funções do `analyzer.py` para inspecionar e extrair os dados, imprime o log de progresso no terminal em tempo real e orquestra a exportação dos dados.
 - (b) `gerarJsonArquivo(...)`: Função responsável por estruturar os dados extraídos de cada arquivo lido (tipo, número de linhas, amostras e ocorrências validadas) e salvar essas informações em documentos `.json` individuais dentro do diretório de saída.
 - (c) `gerarEstatisticas()`: Executada ao final de todo o processo, ela salva o dicionário global de ocorrências do módulo `stats.py` em um documento final chamado `estatisticas.json` e imprime no console o balanço quantitativo completo da execução.

O fluxo de execução do programa é orquestrado pelo módulo `main.py` e segue cinco etapas principais sequenciais:

1. **Inicialização e Busca:** O sistema consulta o `config.py` para saber onde buscar os dados, cria a pasta de destino dos resultados e começa a percorrer a lista de arquivos de entrada.
2. **Leitura e Classificação:** Ao abrir um arquivo, o `analyzer.py` faz uma varredura inicial ignorando linhas em branco para contabilizar o tamanho útil do documento. Em seguida, ele analisa pequenos pedaços do texto para classificar semanticamente do que se trata.
3. **Extração e Validação:** É onde ocorre o filtro dos resultados. O sistema lê o arquivo linha por linha e aplica as Expressões Regulares de captura. Sempre que encontra um possível dado, é acionado o `validators.py` para testar rigorosamente a estrutura daquele padrão, definindo se ele é uma ocorrência válida ou inválida.
4. **Organização e Exportação:** Após varrer todo o documento, o programa normaliza as informações encontradas e gera um arquivo `.json` individual para ele. Esse arquivo contém as estatísticas completas dos arquivos: os dados brutos, os dados extraídos estruturados e os relatórios de inconsistências.
5. **Geração de Estatísticas:** Durante todo o processo das etapas anteriores, o módulo `stats.py` realiza o trabalho de contar cada correspondência encontrada. Quando o último arquivo termina de ser processado, o sistema consolida todas essas métricas globais e exporta o arquivo final `estatisticas.json`, exibindo o balanço quantitativo da operação no terminal.

4. Descrição dos Arquivos Utilizados

O sistema foi submetido a um conjunto de quatro arquivos de entrada obrigatórios, cada um simulando um cenário real de desestruturação de dados. A função de identificação de tipo aplica regras específicas para categorizá-los:

1. `01_atendimentos_bagunçados.txt` (Texto Livre): Arquivo contendo dados sem uma estrutura rígida predefinida. É classificado como Texto Livre por exclusão, quando não atende aos critérios das outras categorias.
2. `02_logs_mistos.log` (Logs): Representa registros operacionais de sistemas. A classificação deste tipo ocorre pela identificação de palavras-chave estruturais no início ou meio das sentenças, mapeadas por Regex, tais como `[INFO]`, `[ERROR]` e `[WARN]`.
3. `03_mensagens_chat.txt` (Chat): Simula históricos de conversas (ex: WhatsApp, Telegram ou chats de suporte). Sua identificação é feita buscando padrões de timestamps estruturados que precedem as mensagens, geralmente no formato `[dd/mm/yyyy hh:mm]`.
4. `04_exportacao_suja.csv` (CSV Inconsistente): Representa dados tabulares exportados com falhas. Sua identificação inicial dá-se pela extensão `.csv`. Diferente dos textos puros, este arquivo passa por uma função específica (`def validarCSV()`) que verifica se a quantidade de colunas em cada linha corresponde ao cabeçalho (utilizando o delimitador `;`) e detecta a presença de campos vazios, gerando um log de inconsistências.

5. Expressões Regulares Desenvolvidas e Justificativas

O núcleo do sistema repousa sobre a mineração textual via Expressões Regulares. Para garantir alta precisão, foi adotada uma estratégia de validação em duas etapas:

1. Captura Genérica: Uma Regex mais permissiva varre a linha em busca de cadeias que se assemelham ao dado procurado (potencialmente inválidas).
2. Validação Estrutural: A cadeia capturada é submetida a uma segunda Regex restritiva, utilizando o método `re.fullmatch()`. Se passar, é classificada como "válida"; caso contrário, como "inválida". Abaixo descrevemos os critérios e justificativas para os principais padrões implementados:

5.1. E-mails

- **Critério de Captura:** Presença do caractere @ cercado por caracteres alfanuméricos.
- **Critério de Validação:** Deve obrigatoriamente possuir um prefixo válido (letras, números, pontos ou sublinhados), seguido do símbolo @, um domínio, e um sufixo (ex: .com, .br).
- **Casos de Invalidez:** Cadeias como `user=9751c7@hotmail.com` (caracteres especiais não permitidos no prefixo) ou `email=@outlook.com` (ausência de prefixo).
 - **Critério de Captura:** Regex mais permissiva para capturar potenciais e-mails mal formatados: `[^; \s@] + @ [^; \s@] +`
 - **Critério de Validação:** Exige prefixo, símbolo @, domínio e sufixo. Regex restritiva utilizada: `[a-zA-Z0-9._%+-] + @ [a-zA-Z0-9.-] + \. [a-zA-Z] {2,}`
 - **Casos de Invalidez:** Cadeias como `user=9751c7@hotmail.com` (caracteres especiais não permitidos no prefixo) ou `email=@outlook.com` (ausência de prefixo).

5.2. Cadastro de Pessoa Física (CPF)

- **Critério de Captura:** Agrupamentos de 11 dígitos numéricos, podendo ou não conter pontos e hifens separadores.
- **Critério de Validação:** O sistema verifica se o CPF está rigorosamente bem formatado no padrão XXX.XXX.XXX-XX. CPFs digitados sem pontuação ou com pontuação em locais errados são capturados, mas classificados como mal formatados (inválidos).
- **Justificativa:** Conforme o escopo do projeto, a validação implementada é puramente sintática (estrutural). Não foi implementada a verificação semântica do dígito verificador matemático.
 - **Critério de Captura:** Agrupamentos numéricos potencialmente inválidos: `\b\d{3} (? : [ . \- ] ? \d{3} ) {2} [ . \- ] ? \d{1,2} \b`
 - **Critério de Validação:** O sistema verifica se o CPF está rigorosamente formatado. Regex utilizada: `\b (? : \d{3} \. \d{3} \. \d{3} - \d{2} | \d{11} ) \b`
 - **Justificativa:** A validação implementada é puramente sintática. Não foi implementada a verificação semântica do dígito verificador matemático.

5.3. Telefones

- **Critério de Validação:** Deve reconhecer padrões nacionais, contemplando o código de área (DDD) entre parênteses, seguido de 8 ou 9 dígitos, com a presença opcional do caractere hífen separando os blocos numéricos.
 - **Critério de Captura:** Captura cadeias com blocos numéricos incorretos:
`\(?\\d{2}\\) ?\\s?\\d{4,5}-?\\d{3,4}`
 - **Critério de Validação:** Deve reconhecer padrões nacionais (DDD opcional + 8 ou 9 dígitos). Regex utilizada:
`\\(?\\d{2}\\) ?\\s?\\d{4,5}-?\\d{4}`

5.4. Datas, Horários e Combinações

- **Datas:** Busca por padrões no formato dd/mm/aaaa ou dd-mm-aaaa, garantindo que os dias limitem-se ao intervalo lógico de 01 a 31, e os meses de 01 a 12.
- **Horários:** Busca pelo padrão hh:mm ou hh:mm:ss, limitando as horas a 23 e minutos/segundos a 59.
- **Combinações:** Uma Regex composta identifica instâncias onde data e hora aparecem conjugadas de forma sequencial na mesma cadeia.
 - **Datas:** Busca pelo formato estruturado com a Regex:
`\\d{2}/\\d{2}/\\d{4}`
 - **Horários:** Busca pelo padrão de horas e minutos com a Regex:
`\\d{2}:\\d{2}(?::\\d{2})?`
 - **Combinações:** Uma Regex composta identifica instâncias onde data e hora aparecem conjugadas na mesma cadeia:
`\\d{2}/\\d{2}/\\d{4}\\s\\d{2}:\\d{2}(?::\\d{2})?`

5.5. URLs e Valores Monetários

- **URLs:** A validação exige a presença dos protocolos http:// ou https://, seguidos de um domínio de topo estruturado.
- **Valores Monetários:** Captura estritamente valores formatados em moeda brasileira, exigindo a presença do prefixo R\$, permitindo pontos como separadores de milhar e exigindo a vírgula para a separação dos centavos (ex: R\$ 1.500,00).
 - **URLs (Captura):** Varredura inicial utilizando:
`(?:https?:/[\\^\\s;]+|www\\. [\\^\\s;]+)`
 - **URLs (Validação):** A validação exige o protocolo rigoroso utilizando:
`https?:/[\\^\\s]+`
 - **Valores Monetários:** Exige a presença do prefixo R\$ e formatação de milhar/centavos. Regex utilizada:
`R\\$\\s?\\d{1,3}(?:\\.\\d{3})*,\\d{2}`

5.6. Nomes Próprios

- **Justificativa e Critérios:** A extração de nomes próprios em textos desestruturados é um desafio clássico de NLP (Processamento de Linguagem Natural), pois depende fortemente do contexto semântico. Para este projeto baseado estritamente em linguagens regulares, definiu-se que um nome próprio válido consistiria em palavras compostas, onde cada bloco inicia com letra maiúscula seguida de letras minúsculas.

- Filtros adicionais: Para evitar a captura de inícios de frases normais, a Regex foi ajustada para buscar ocorrências de pelo menos duas palavras capitalizadas em sequência (Ex: João Silva), ignorando palavras isoladas no início de sentenças.
 - **Justificativa e Critérios:** Foi definido que um nome próprio consiste em palavras compostas com inicial maiúscula. Regex utilizada:
`\b[A-ZÁÉÍÓÚÂÊÔÃÕÇ][a-záéíóúâêôãõç]+(?:\s[A-ZÁÉÍÓÚÂÊÔÃÕÇ][a-záéíóúâêôãõç]+)+`
 - **Filtros adicionais:** A Regex foi ajustada para exigir pelo menos duas palavras em sequência, ignorando palavras isoladas no início de sentenças.

6. Testes Experimentais e Análise dos Resultados

O sistema foi executado em ambiente terminal via comando `python3 main.py`. Durante a execução, a interface de linha de comando imprimiu logs em tempo real, evidenciando o arquivo processado, o tipo identificado, o total de linhas contabilizadas e uma amostra abreviada (função `def mostrarAmostra()`) garantindo a visibilidade da leitura sem sobrecarregar o console.

6.1. Saídas Estruturadas (JSON)

Para cada arquivo lido no diretório `./assets/`, o sistema gerou um documento correspondente na pasta `./resultados_json/`. Cada ocorrência extraída foi mapeada contendo sua localização e classificação, conforme a estrutura abaixo:

```
"email": {
  "linha": 10,
  "arquivo_origem": "01_atendimentos_bagunçados.txt",
  "valor": "usuario@email.com",
  "classificacao": "valido"
}
```

No caso específico do arquivo `04_exportacao-suja.csv`, além da extração de padrões consolidados, o exportador estruturou um vetor paralelo denominado `inconsistencias_csv`. Este vetor age como um log de auditoria, apontando precisamente as linhas que possuíam colunas em branco ou número divergente de separadores em relação ao cabeçalho principal, conforme ilustrado no fragmento a seguir:

```
"inconsistencias_csv": [
  {
    "linha": 42,
    "problema": "Quantidade de colunas inconsistente",
    "quantidade_colunas": 3,
    "esperado": 5,
    "conteudo": "João Silva;joao@email.com;99999-0000"
  },
  {
    "linha": 45,
    "coluna": 2,
    "problema": "Campo vazio",
  }
]
```



```

        "conteudo": "Maria Souza;maria@email.com;;Ativo"
    }
]

```

6.2. Análise Quantitativa Global

A execução culminou na geração do arquivo estatisticas.json. O dicionário global em memória demonstrou com sucesso a distribuição das ocorrências. Foi possível observar que o arquivo de Texto Livre continha a maior concentração de dados desformatados (inválidos), enquanto o arquivo de Logs apresentou alta padronização estrutural de datas e horários, porém com ocorrências dispersas de IPs e URLs.

6.3. Limitações Observadas

Apesar do alto índice de captura, os testes evidenciaram as limitações inerentes ao uso exclusivo de expressões regulares:

1. A validação do CPF foi apenas estrutural (não descobre CPFs formalmente corretos, mas matematicamente inexistentes).
2. URLs não puderam ser validadas semanticamente (não há como garantir via Regex se o link está ativo ou quebrado).
3. A ausência de normalização prévia de encoding e o uso de uma quebra manual de delimitadores para o arquivo CSV, ao invés de um parser formal, pode gerar falhas caso o caractere ; seja utilizado no meio de um texto escapado por aspas.

7. Instruções de Execução

Para a correta execução do sistema, é necessário que o ambiente computacional possua a linguagem Python (recomenda-se a versão 3.10 ou superior) previamente instalada. Todos os módulos utilizados para o processamento, mineração e estruturação dos dados (como re, json e pathlib) integram a biblioteca padrão do Python. Isso garante alta portabilidade, pois dispensa o uso de gerenciadores de pacotes (como o pip) e a instalação de quaisquer dependências de terceiros.

O processo de execução segue essas etapas:

1. Download: O código-fonte e os arquivos de teste devem ser baixados através do repositório oficial do projeto (https://github.com/lucaszost/Projeto_de_Automatos).
2. Extração: Após o término do download, extraia o conteúdo do arquivo compactado em um diretório arbitrário.
3. Navegação: Em seguida, abra o terminal do seu sistema operacional e navegue, utilizando o comando cd, até a pasta raiz onde os arquivos do projeto foram extraídos.
4. Execução: Por fim, inicie o orquestrador do sistema inserindo e executando o comando "python3 main.py" no terminal ou através de uma IDE, como o Visual Studio Code.

8. Principais Dificuldades

Durante o planejamento e a melhoria da estrutura do código, foram encontradas algumas dificuldades técnicas na impressão e filtração do conteúdo desejado. Dentre elas:

1. O conteúdo impresso após a compilação do algoritmo não retornava as ocorrências inválidas, somente as ocorrências válidas.
2. A estrutura do output não tornava as informações visivelmente organizadas. A falta de contraste dificultava a compreensão do texto de saída da interface do programa para o usuário.
3. A falta de divisão de funções do `main.py` impossibilitava a manutenção e a organização da árvore do programa de forma extensa, que posteriormente foi corrigido.

9. Comentários Finais

O desenvolvimento deste projeto comprovou a eficácia das Expressões Regulares como ferramenta primária de mineração e validação sintática. O sistema atendeu a todos os requisitos propostos, sendo capaz de absorver arquivos caóticos, classificá-los, extrair informações cruciais e estruturá-las em um formato padronizado (JSON) amplamente aceito no mercado de tecnologia.

A estratégia de validação em duas etapas (permissiva e restritiva) mostrou-se particularmente inteligente para auditorias de dados, pois permite não apenas ignorar lixo eletrônico, mas catalogar ativamente o dado que foi inserido de forma equivocada pelo usuário (como e-mails com sintaxe incorreta).

Como trabalhos futuros, visando superar as limitações do sistema atual, sugere-se a implementação de funções em Python para a validação matemática de documentos (CPFs, CNPJs), o uso de requisições web para checagem de integridade de URLs e a integração de modelos de NLP (como a biblioteca `spaCy`) para o Reconhecimento de Entidades Nomeadas (NER), garantindo uma extração infalível de Nomes Próprios independente da capitalização do texto original.

Em suma, este trabalho reafirma a importância vital das bases teóricas da computação. Demonstra-se que conceitos matemáticos e abstratos de Linguagens Formais e Autômatos não são apenas objetos de estudo acadêmico, mas sim tecnologias indispensáveis para resolver problemas reais de desestruturação de dados, unindo o rigor científico à aplicação prática na engenharia de software contemporânea.

10. Referências

1. HOPCROFT, John E.; MOTWANI, Rajeev; ULLMAN, Jeffrey D. Introdução à Teoria de Autômatos, Linguagens e Computação. 2. ed. Rio de Janeiro: Elsevier, 2002.
2. SIPSER, Michael. Introdução à Teoria da Computação. 2. ed. São Paulo: Cengage Learning, 2007.
3. FRIEDL, Jeffrey E. F. Mastering Regular Expressions. 3. ed. Sebastopol, CA: O'Reilly Media, 2006.
4. PYTHON SOFTWARE FOUNDATION. Documentação oficial da biblioteca `'re'` (Regular expression operations). Disponível em: <https://docs.python.org/3/library/re.html>. Acesso em: 22 maio 2026.